

**Université de Genève**  
Faculté des Sciences — Département d'informatique

# Projet de Bachelor

Année académique 2024–2025

---

**Génération de formule SAT difficile et compacte**

---

**Encadrant**  
Prof. Pierre Leone

**Étudiant**  
Loïc Bardi



**UNIVERSITÉ  
DE GENÈVE**

---

**FACULTÉ DES SCIENCES**  
Département d'informatique

## Abstract

L'objectif initial de ce projet était de concevoir une formule SAT difficile — nous reviendrons plus en détail sur ce que nous entendons par difficile — puis de chercher à en **réduire le nombre de variables**, afin de limiter sa taille tout en conservant sa complexité à être résolue par un solveur SAT.

Au fil du projet, certains questionnements et observations annexes ont naturellement émergé. Cela nous a conduit à emprunter quelques détours, afin d'explorer ces problématiques intermédiaires et d'enrichir notre compréhension du sujet.

# Contents

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                              | <b>3</b>  |
| <b>2</b> | <b>État de l'art</b>                                             | <b>3</b>  |
| 2.1      | Formules SAT difficiles : définitions et concepts clés . . . . . | 3         |
| 2.2      | Méthodes de génération de formules difficiles . . . . .          | 5         |
| 2.3      | Algorithmes de résolution . . . . .                              | 5         |
| <b>3</b> | <b>Modélisation logique du problème de factorisation</b>         | <b>7</b>  |
| 3.1      | Problème FACT : définition formelle . . . . .                    | 7         |
| 3.2      | Modélisation en circuit logique . . . . .                        | 7         |
| <b>4</b> | <b>Transformation en formule SAT</b>                             | <b>9</b>  |
| 4.1      | Transformation de Tseitin . . . . .                              | 9         |
| 4.2      | Application . . . . .                                            | 10        |
| 4.3      | Implémentation de la génération CNF d'une multiplication . . . . | 10        |
| <b>5</b> | <b>Résolution avec Z3</b>                                        | <b>12</b> |
| 5.1      | Principe de la résolution SAT . . . . .                          | 12        |
| 5.2      | Métriques de complexité considérées . . . . .                    | 12        |
| 5.3      | Résultats expérimentaux . . . . .                                | 13        |
| <b>6</b> | <b>Analyse structurelle des formules générées</b>                | <b>14</b> |
| 6.1      | Répartition des clauses . . . . .                                | 14        |
| 6.2      | Taille totale et ratio clause/variable . . . . .                 | 15        |
| 6.3      | Analyse du ratio clause/variable . . . . .                       | 16        |
| <b>7</b> | <b>Réduction de la taille de la formule</b>                      | <b>18</b> |
| 7.1      | Règles de simplification élémentaires . . . . .                  | 18        |
| 7.2      | Propagation unitaire . . . . .                                   | 20        |
| 7.3      | Réduction par assignation de variables . . . . .                 | 21        |
| <b>8</b> | <b>Conclusion</b>                                                | <b>23</b> |

# 1 Introduction

Les problèmes de satisfiabilité booléenne (SAT), et en particulier le  $k$ -SAT, occupent une place centrale dans la théorie de la complexité. Il s'agit de déterminer si une formule booléenne, généralement donnée sous forme normale conjonctive ( $k$ -CNF), admet une assignation des variables qui la rend vraie. Ce problème est fondamental, à la fois pour des raisons théoriques —  $k$ -SAT pour  $k \geq 3$  est l'un des premiers problèmes prouvés NP-complets — et pour ses nombreuses applications pratiques.

Malgré les progrès réalisés dans le développement de solveurs SAT, il est aujourd'hui encore difficile de prédire, pour une formule donnée, si elle sera "facile" ou "difficile" à résoudre. C'est pourquoi la génération de formules SAT difficiles, mais bien structurées, constitue un enjeu important pour mieux comprendre le comportement de ces solveurs, et tester leurs limites empiriques.

Dans ce contexte, nous avons choisi de nous appuyer sur un problème bien connu pour sa difficulté algorithmique : la factorisation d'un entier composite  $x = p \cdot q$ , où  $p$  et  $q$  sont deux grands nombres premiers. Ce problème — au cœur de nombreux systèmes cryptographiques comme RSA — est réputé pour être difficile à résoudre en pratique.

L'idée sous-jacente était que si l'on modélise un problème reconnu comme difficile, que l'on réduit ce problème à SAT, alors les formules SAT obtenues le seraient vraisemblablement aussi et nous pourrions ainsi les utiliser pour nos expérimentations. Bien que cela ne garantisse pas systématiquement une difficulté équivalente — certaines modélisations peuvent produire des instances triviales à résoudre malgré la complexité théorique du problème initial — nous avons estimé que la structure intrinsèquement complexe de la factorisation pouvait raisonnablement conduire à des formules SAT exigeantes.

De plus, la simplicité conceptuelle du problème, la facilité à générer des instances (par simple multiplication de deux nombres premiers), la modélisation naturelle du problème à l'aide de circuits logiques ainsi que l'existence des transformations de Tseytin, nous ont semblé des arguments solides pour adopter cette approche.

Nous avons choisi une approche directe — voire naïve — en modélisant explicitement la multiplication binaire, puis en étudiant expérimentalement différentes techniques de simplification et de réduction. Si cette méthode permet moins de contrôle théorique sur la taille initiale, elle offre une plus grande flexibilité pour explorer les effets de simplifications successives, et pourrait, à terme, être combinée avec des techniques de modélisation plus avancées.

## 2 État de l'art

### 2.1 Formules SAT difficiles : définitions et concepts clés

Une formule 3-SAT est dite *difficile* lorsqu'elle nécessite un effort computationnel élevé de la part des solveurs, même les plus modernes. Plusieurs indicateurs

empiriques ont été proposés pour caractériser ces formules :

- **Backbones:** Le backbone d'une formule satisfaisable est l'ensemble des littéraux vrais dans toutes les solutions de la formule. Un backbone étendu rend les décisions plus sensibles aux erreurs d'affectation. En effet, plus le backbone est étendu, plus il y a d'occasions de faire un choix erroné lors de la décision d'une variable, ce qui force de nombreux backtracks. Kilby et al. (2005) [Kil+05] ont constaté empiriquement une corrélation entre la taille du backbone et la difficulté : les instances à backbone large tendent à être plus ardues pour les algorithmes de type DPLL que celles à backbone réduit. Notons que déterminer si une formule a un backbone non vide est un problème NP-complet.
- **Backdoors:** Une backdoor est un petit ensemble de variables telles que, une fois assignées d'une certaine façon, le reste de la formule devient facilement résoluble (par exemple par simplification en propagation unitaire). Autrement dit, un solveur qui devinerait correctement les valeurs de ces variables réduirait considérablement la difficulté du problème à un cas trivial. Les formules possédant de petits backdoors peuvent donc, en théorie, être résolues rapidement en trouvant la bonne combinaison pour ces variables critiques. Cependant, ces backdoors sont difficiles à identifier : Williams et al. (2003) [WGS03] ont formalisé cette notion et montré qu'en général, trouver la plus petite backdoor est NP-dur. De plus, les instances purement aléatoires semblent dépourvues de backdoors de petite taille : par exemple, pour une formule 3-SAT aléatoire, tout ensemble de backdoor candidate représente typiquement une fraction non négligeable des variables, et non un petit sous-ensemble indépendant.
- **Seuil de phase:** Le seuil (ou point) de transition désigne la densité de contraintes à laquelle une formule passe abruptement de satisfiable à non satisfiable. Ce phénomène, mis en évidence par les travaux de Hogg et al. (1996) [HHW96] et d'autres, s'accompagne d'une explosion du temps de résolution nécessaire à proximité du seuil. Mitchell *et al.* [MSL+92] ont observé que la difficulté culmine lorsque le ratio clauses/variables avoisine un seuil critique (environ 4,26), point auquel la probabilité de satisfiabilité chute brutalement.
- **Complexité arborescente:** Cette métrique mesure la mémoire minimale requise par une preuve de résolution arborescente de l'insatisfiabilité de la formule, c'est-à-dire une preuve où chaque clause dérivée est utilisée au plus une fois (pas de réutilisation, formant une structure d'arbre). Ansótegui et al. (2008) [Ans+08] ont proposé la tree-like resolution space complexity comme un indicateur quantitatif de la dureté pratique d'une formule pour les solveurs basés sur DPLL. En effet, un solveur DPLL sans apprentissage de clauses produit essentiellement une preuve de résolution arborescente: la taille de la plus petite preuve borne donc la complexité de l'instance pour ce type de solveur. Des travaux ultérieurs (Järvisalo et

al. 2012 [Jär+12] ) ont confirmé empiriquement que cette mesure était un prédicteur plus fin de la difficulté d'une formule que d'autres mesures théoriques.

## 2.2 Méthodes de génération de formules difficiles

**Approches aléatoires.** Les générateurs de formules 3-SAT aléatoires ajustent le ratio clauses/variables pour cibler la transition de phase. Achlioptas *et al.* ont notamment proposé une génération de formules satisfiables proches du seuil, maximisant ainsi la difficulté. Néanmoins, ces instances sont souvent trop homogènes, ce qui limite leur représentativité des problèmes réels.

**Approches structurées.** D'autres méthodes s'appuient sur des réductions depuis des problèmes réputés difficiles, comme FACT par exemple. Factoriser n'est pas prouvé NP-difficile, mais en pratique c'est un problème extrêmement ardu pour les entiers de grande taille. On peut donc transformer une instance de factorisation en une formule SAT, de sorte que toute affectation satisfaisante de la formule encode une factorisation valide du nombre d'entrée. Horie et Watanabe [HW97] ont utilisé le théorème des restes chinois pour transformer une instance de factorisation en CNF. Cette approche leur ont permis de produire des formules SAT à la fois satisfiables et difficiles, tout en contrôlant précisément leur taille. Leur algorithme permet de construire, par exemple, une formule 3-SAT d'environ 5600 variables représentant la factorisation d'un entier de 100 bits. La résolution de cette formule est aussi difficile que la factorisation du nombre correspondant. Bebel et Yuen [BY13] ont quant à eux développé *ToughSAT*, un générateur de formules encodant explicitement la multiplication binaire, produisant des formules particulièrement résistantes aux solveurs.

**Approches hybrides.** Récemment, Li *et al.* [Li+23] ont introduit *HardSAT-Gen*, un générateur de formules à la fois dures et structurellement proches d'instances industrielles. Il s'appuie sur des mécanismes de génération neuronaux capables de préserver des caractéristiques telles que la modularité ou la distribution non uniforme des variables.

## 2.3 Algorithmes de résolution

**DPLL.** L'algorithme DPLL (Davis–Putnam–Logemann–Loveland ) [DLL62] explore récursivement l'espace des affectations. En simplifiant, DPLL part d'une assignation partielle vide et choisit récursivement une variable non assignée, lui affecte successivement la valeur Vrai ou Faux, puis poursuit ainsi jusqu'à trouver soit un modèle complet satisfaisant toutes les clauses, soit une contradiction qui provoque un retour en arrière. S'il atteint une feuille d'arbre où une clause est falsifiée (conflit), l'algorithme remonte au dernier choix non exploré pour essayer l'autre branche (backtrack). À l'inverse, si une assignation totale satisfaisante est construite, la formule est satisfiable et le processus s'arrête. En l'absence de solution (dans le pire cas), l'exploration finit par énumérer toutes les valuations

possibles et conclut à l'insatisfiabilité de la formule lorsque toutes les branches ont mené à un conflit. Cet algorithme est fortement impacté par la taille des backbones et l'absence de backdoors, qui entraînent une exploration coûteuse. Certaines formules sont intraitables pour DPLL en raison de leur structure arborescente profonde [Ans+08].

Ci-dessous le pseudo-code de l'algorithme:

---

**Algorithm 1:** DPLL (Davis–Putnam–Logemann–Loveland)

---

**Input:** Un ensemble de clauses  $\Phi$   
**Output:** **true** si  $\Phi$  est satisfiable, **false** sinon

```

1 Function DPLL( $\Phi$ ):
    // Propagation unitaire
2   while il existe une clause unitaire  $\{l\}$  dans  $\Phi$  do
3      $\Phi \leftarrow \text{unit\_propagate}(l, \Phi)$ ;
    // Élimination des littéraux purs
4   while il existe un littéral pur  $l$  dans  $\Phi$  do
5      $\Phi \leftarrow \text{pure\_literal\_assign}(l, \Phi)$ ;
    // Conditions d'arrêt
6   if  $\Phi$  est vide then
7     return true;
8   if  $\Phi$  contient une clause vide then
9     return false;
    // Choix récursif
10   $l \leftarrow \text{choose\_literal}(\Phi)$ ;
11  return DPLL( $\Phi \cup \{l\}$ ) or DPLL( $\Phi \cup \{\neg l\}$ );

```

---

**CDCL.** Le CDCL (Conflict-Driven Clause Learning) est une amélioration de l'algorithme DPLL. Il apprend de nouvelles clauses à partir des conflits rencontrés et utilise un retour arrière non chronologique pour éviter de refaire les mêmes erreurs [Oh15]. Cela lui permet d'explorer l'espace de recherche de manière plus intelligente, en évitant les répétitions inutiles. Cependant, cette approche peut consommer beaucoup de mémoire, surtout quand les formules sont très denses ou contiennent des clauses fortement liées [Jär+12].

### 3 Modélisation logique du problème de factorisation

#### 3.1 Problème FACT : définition formelle

##### Définition du problème FACT

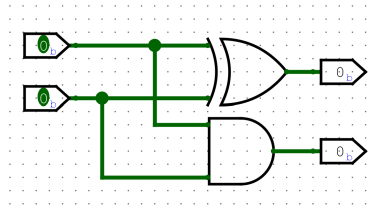
Étant donné un entier  $x \in \mathbb{N}$  composite, le problème de factorisation (FACT) consiste à retrouver deux entiers premiers  $p$  et  $q$  tels que  $x = p \cdot q$ . Seul le produit  $x$  est fourni en entrée. L'objectif est donc de retrouver les deux facteurs premiers.

#### 3.2 Modélisation en circuit logique

Afin de rendre le problème de factorisation exploitable par un solveur SAT, il est nécessaire de le modéliser sous forme logique afin de pouvoir y appliquer les transformations de Tseytin. Pour cela, nous avons choisi d'encoder l'opération de multiplication binaire au moyen de circuits logiques, constitués principalement de portes AND et d'additionneurs.

Le cœur de cette modélisation repose sur deux composants fondamentaux :

- **HALF-ADDER:**



| $a$ | $b$ | $x$ | $y$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 0   |
| 0   | 1   | 1   | 0   |
| 1   | 0   | 1   | 0   |
| 1   | 1   | 0   | 1   |

Figure 1: Demi-additionneur avec sa table de vérité



• **FULL-ADDER:**

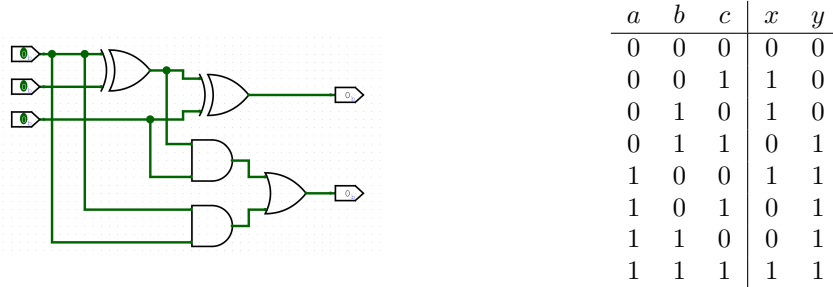


Figure 2: Additionneur complet avec sa table de vérité

Combinés, ces deux circuits permettent d'additionner deux nombres binaires. En effet, pour effectuer une addition binaire, il est nécessaire d'utiliser un demi-additionneur pour le bit de poids faible, suivi de  $n-1$  additionneurs complets, où  $n$  est la taille du plus grand des deux opérandes. La différence principale entre ces deux circuits réside dans le nombre d'entrées : le demi-additionneur prend uniquement deux bits (les bits correspondants de chaque opérande), tandis que l'additionneur complet en prend trois, afin de gérer une éventuelle retenue provenant de l'addition des bits précédents. L'addition des deux bits de poids faible est la seule pour laquelle on est assuré qu'aucune retenue n'intervient en entrée.

Une multiplication binaire peut être vue comme une suite d'additions en cascade. Si l'on note  $A$  et  $B$  les deux opérandes de la multiplication, alors pour chaque bit de  $B$ , on effectue une multiplication entre ce bit et tous les bits de  $A$ , ce qui donne un produit partiel binaire. Chacun de ces produits partiels est ensuite décalé (en fonction de la position du bit dans  $B$ ) puis additionné aux autres. On obtient ainsi une accumulation d'additions successives, étage par étage, pour produire le résultat final.

Il est important de noter que pour réaliser une multiplication bit à bit, une simple porte logique AND suffit. En effet, le produit de deux bits binaires correspond exactement au résultat d'un ET logique entre ces deux bits.

Le circuit logique correspondant à la multiplication peut être décomposé en trois parties, chacune représentant un étage de la cascade : le premier niveau, les étages intermédiaires, et le dernier niveau. Voici une représentation de ces trois composantes du circuit :

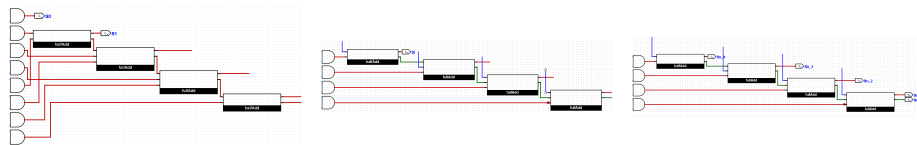


Figure 3: Structure en cascade d'un circuit de multiplication binaire : étage initial, intermédiaires et final

## 4 Transformation en formule SAT

### 4.1 Transformation de Tseytin

La transformation de Tseytin permet de convertir un circuit logique en une formule booléenne équisatisfiable en forme normale conjonctive (CNF). Cette transformation introduit une nouvelle variable pour chaque porte logique, ce qui permet de préserver l'équivalence de satisfiabilité tout en gardant la taille de la formule linéaire par rapport à celle du circuit d'origine. Voici ci-dessous la formule SAT qui correspond à chaque porte logique:

| Type                                                                                            | Operation                   | CNF sub-expression                                                                                                                 |
|-------------------------------------------------------------------------------------------------|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------|
|  <b>AND</b>    | $C = A \cdot B$             | $(\bar{A} \vee \bar{B} \vee C) \wedge (A \vee \bar{C}) \wedge (B \vee \bar{C})$                                                    |
|  <b>NAND</b>   | $C = \overline{A \cdot B}$  | $(\bar{A} \vee \bar{B} \vee \bar{C}) \wedge (A \vee C) \wedge (B \vee C)$                                                          |
|  <b>OR</b>     | $C = A + B$                 | $(A \vee B \vee \bar{C}) \wedge (\bar{A} \vee C) \wedge (\bar{B} \vee C)$                                                          |
|  <b>NOR</b>    | $C = \overline{A + B}$      | $(A \vee B \vee C) \wedge (\bar{A} \vee \bar{C}) \wedge (\bar{B} \vee \bar{C})$                                                    |
|  <b>NOT</b>  | $C = \bar{A}$               | $(\bar{A} \vee \bar{C}) \wedge (A \vee C)$                                                                                         |
|  <b>XOR</b>  | $C = A \oplus B$            | $(\bar{A} \vee \bar{B} \vee \bar{C}) \wedge (A \vee B \vee \bar{C}) \wedge (A \vee \bar{B} \vee C) \wedge (\bar{A} \vee B \vee C)$ |
|  <b>XNOR</b> | $C = \overline{A \oplus B}$ | $(\bar{A} \vee \bar{B} \vee C) \wedge (A \vee B \vee C) \wedge (A \vee \bar{B} \vee \bar{C}) \wedge (\bar{A} \vee B \vee \bar{C})$ |

Figure 4: Transformation de Tseytin par porte logique [Wik25c]

## 4.2 Application

Grâce à la modélisation du circuit de multiplication et à la transformation de Tseytin, nous avons pu implémenter un algorithme capable de produire une formule SAT représentant une opération de multiplication.

Quelques précisions sont nécessaires concernant le fonctionnement de cet algorithme :

- **Choix de la taille des entrées** : selon le design du circuit multiplieur utilisé, il est indispensable de connaître à l'avance la taille (en bits) des entrées  $p$  et  $q$  pour pouvoir générer le circuit logique, et donc appliquer la transformation de Tseytin. Par souci de simplicité et de flexibilité, l'algorithme prend cette taille comme paramètre d'entrée. En effet, l'objectif de notre projet n'est pas de résoudre le problème de factorisation en tant que tel, mais bien de générer une formule CNF. À noter que, dans tous les cas, on peut choisir comme taille pour  $p$  et  $q$  celle de  $x$  (qui est connu dans le problème FACT), ce qui garantit que toutes les solutions possibles soient couvertes. Cependant, cette approche alourdit la formule et allonge les temps de résolution.
- **Indépendance par rapport à la valeur de  $x$**  : il est important de souligner que la génération du circuit logique de [multiplication](#) ne dépend que de la taille des entrées  $p$  et  $q$ . La valeur de  $x$  n'intervient qu'une fois le circuit transformé en formule CNF, en venant fixer les valeurs des variables de sortie représentant les bits du produit.
- **Format de sortie** : le format *DIMACS* a été choisi pour représenter les formules générées. Il s'agit du format standard utilisé dans la littérature pour encoder les formules CNF. Un parseur a donc été développé afin de convertir les formules produites en fichiers au format *DIMACS*.
- **Gestion des variables** : la transformation de Tseytin introduit un grand nombre de variables intermédiaires. Il a donc été nécessaire de mettre en place une gestion rigoureuse de ces variables afin de pouvoir identifier et retrouver précisément celles représentant les entrées  $(p, q)$  et la sortie  $x$  du circuit.

## 4.3 Implémentation de la génération CNF d'une multiplication

On construit progressivement la formule logique en concaténant les fragments CNF générés par chaque sous-composant logique, à l'aide de la transformation de Tseytin.

L'extrait ci-dessous illustre la méthode **MULT**, responsable de la génération des clauses correspondant au circuit de multiplication. Pour chaque bit de  $p$ , cette fonction effectue des multiplications bit-à-bit avec les bits de  $q$ , puis enchaîne les additions des produits partiels à l'aide de demi-additionneurs et

d'additionneurs complets. Les sorties sont stockées dans un vecteur  $x$  représentant le produit.

NOTE: Attention dans le code  $p$  et  $q$  correspondent à  $x$  et  $y$  et  $x$  correspond à  $z$ .

```

1 def MULT(self, size_x: int, size_y: int) -> str:
2     """
3     Ajoute a la variable 'sat' la formule CNF de  $x * y = z$ 
4     Args:
5         size_x, size_y (int):
6             taille des entrees x et y en bits
7     """
8
9
10    X, Y = [f"x{i}" for i in range(size_x)], [f"y{i}" for i in
11    range(size_y)]
12    Z = []
13
14    self.sat += self.AND(X[0], Y[0]) # "etage 0": premiere
15    multiplication dont on recupere directement le resultat en
16    sortie
17
18    for i in range(size_y-1): # correspond a l'indice des B
19        for j in range(size_x): # correspond a l'indice des A
20            if i == 0: # 1ere etage
21                ... [comportement legerement different]
22            else:
23                m = self.AND(X[j], Y[i+1])
24                g = f"gate{self.i-1}"
25                res, n, c_out = self.half_ADD(Z[j], g) if j == 0
26            else self.full_ADD(Z[j], g, c_out)
27                Z[j] = n
28                if j == size_x - 1:
29                    Z.append(c_out)
30
31            self.sat += " * " + m + " * " + res
32
33    Z.pop(0) #permet de supprimer le resultat qui sera la
34    sortie et donc plus necessaire pour la suite des calculs

```

Listing 1: Extrait de la méthode MULT utilisée pour la génération de la formule CNF

Le code complet de cette méthode ainsi que celui des méthodes appelées est disponible sur gitlab [Bar25].

Nous disposons désormais de formules SAT représentant des multiplications entières binaires. Nous allons introduire maintenant la manière dont celles-ci sont résolues.

## 5 Résolution avec Z3

### 5.1 Principe de la résolution SAT

Les formules obtenues par notre transformation correspondent à des instances du problème de satisfiabilité booléenne (**SAT**). Ce problème consiste à déterminer s'il existe une affectation des variables booléennes (vrai ou faux) telle que la formule entière soit évaluée comme vraie.

**SAT** est le problème de base d'un large éventail de problèmes en logique et en vérification formelle. Plus précisément, lorsque les formules sont exprimées en forme normale conjonctive (**CNF**) et que chaque clause contient au plus  $k$  littéraux, on parle de  $k$ -**SAT** [Wik25a]. Il est bien établi que le problème  $k$ -**SAT** est NP-complet dès que  $k \geq 3$ .

Cela signifie que, sauf preuve du contraire, aucun algorithme connu ne permet de résoudre toutes les instances de  $k$ -**SAT** en temps polynomial. En pratique, certains solveurs parviennent à résoudre efficacement de nombreuses instances, mais la complexité peut croître de façon exponentielle dans le pire des cas.

Un solveur **SAT** est un algorithme dont l'objectif est de déterminer s'il existe une affectation des variables qui satisfait une formule donnée, ou de prouver qu'aucune n'existe. Si une telle affectation existe, la formule est dite **SAT**; sinon, elle est **UNSAT**. Une formule **SAT** peut admettre une ou plusieurs affectations satisfaisantes.

Dans notre cas, chaque formule encode un problème de multiplication où les facteurs premiers sont fixés : il n'existe donc qu'une unique solution satisfaisante par construction.

Dans ce travail, nous avons utilisé le solveur **Z3** [MB24], développé par Microsoft Research, via son interface Python. Ce choix s'est imposé pour sa simplicité d'intégration, sa relative stabilité, et sa prise en charge du format standard **DIMACS** dans lequel nos formules sont exprimées. Toutefois, la documentation disponible s'est parfois révélée lacunaire lors de l'utilisation avancée de certaines fonctionnalités.

### 5.2 Métriques de complexité considérées

Afin d'évaluer la difficulté des formules générées et de pouvoir les comparer entre elles, nous nous sommes principalement appuyés sur deux métriques fournies par l'objet `Statistics` de l'API Python de **Z3** :

- le temps de résolution, mesuré en secondes, accessible via le champ `time` ;
- le nombre de backjumps, accessible via le champ `backjumps`.

D'autres indicateurs intéressants sont également disponibles, tels que le nombre de conflits, de décisions ou de propagations effectuées par le solveur. Toutefois, ces métriques n'ont pas été exploitées dans le cadre de ce rapport.

Le nombre de *backjumps* représente le nombre de retours en arrière effectués par le solveur lors de la recherche d'une solution. Chaque backjump reflète un point où le solveur, confronté à un conflit logique, abandonne plusieurs décisions successives pour revenir à un niveau antérieur de la pile de décisions. Ce compteur constitue ainsi un bon indicateur de la complexité rencontrée pendant la résolution.

### 5.3 Résultats expérimentaux

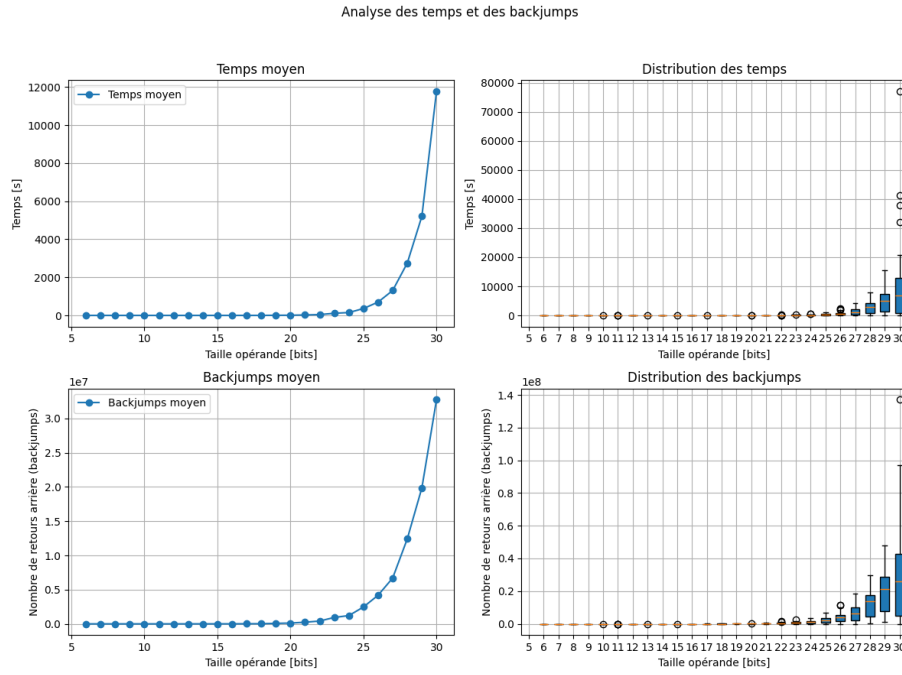


Figure 5: Analyse expérimentale de la complexité : en haut, le temps de résolution moyen (gauche) et sa distribution (droite) ; en bas, le nombre moyen de backjumps (gauche) et sa distribution (droite), en fonction de la taille des opérandes  $p$  et  $q$

Nous avons cherché à évaluer empiriquement le comportement du solveur Z3 en fonction de la taille des entiers  $p$  et  $q$ , noté  $k$ . Pour chaque taille de bits  $k$  (allant de 5 à 30), nous avons généré et résolu 30 instances indépendantes.

Les courbes montrent clairement une croissance exponentielle du temps de résolution ainsi que du nombre de backjumps, deux métriques fortement corrélées. À partir de  $k = 25$ , la difficulté explose : pour  $k = 30$ , le temps moyen dépasse 12 000 secondes, soit plus de 3h20. À titre de comparaison, les instances de taille  $k \leq 20$  sont résolues en quelques secondes.

Les boxplots de droite révèlent également une forte disparité entre instances de même taille : pour  $k = 30$ , certaines formules sont résolues en quelques heures, tandis qu’une instance extrême a nécessité près de 80 000 secondes, soit plus de 22 heures de calcul. Cette variabilité suggère que le comportement du solveur est fortement influencé par la structure interne des formules, malgré leur taille identique.

Compte tenu de cette explosion du temps de résolution, il ne semble pas pertinent, dans le cadre de ce projet, d’étendre nos expériences à des instances plus grandes que celles issues de multiplications sur 30 bits. Bien que 30 bits ne représente que des entiers de l’ordre du milliard — ce qui reste modeste comparé aux standards industriels — ces instances sont déjà suffisamment exigeantes pour mettre en évidence les phénomènes de complexité que nous souhaitons analyser.

Plutôt que de viser des cas d’instances inaccessibles, nous nous sommes plutôt concentrés sur l’évolution de la difficulté à résoudre les formules selon les transformations appliquées ou les modifications structurelles effectuées. Si les résultats expérimentaux obtenus sur les petites instances s’avèrent concluants, ils pourraient être généralisés à des tailles plus grandes.

## 6 Analyse structurelle des formules générées

### 6.1 Répartition des clauses

Les formules obtenues sont constituées de clauses à deux et à trois littéraux. La répartition résulte directement des choix effectués lors de la [modélisation logique](#) du circuit de multiplication. Cette répartition des clauses est définie par les deux équations suivantes, exprimées en fonction de  $k$ , la taille (en bits) des opérandes  $p$  et  $q$  :

1. Nombre de clauses à deux littéraux :  $8 \cdot k^2 - 10 \cdot k$
2. Nombre de clauses à trois littéraux :  $12 \cdot k^2 - 17 \cdot k$

Ces équations ont été établies à partir de la structure du circuit logique utilisé, en tenant compte de la traduction en formules SAT obtenue par la [transformation de Tseytin](#). Chaque porte logique génère un nombre fixe de clauses selon sa nature :

- **AND** : 2 clauses à 2 littéraux, 1 clause à 3 littéraux
- **OR** : 2 clauses à 2 littéraux, 1 clause à 3 littéraux
- **XOR** : 0 clause à 2 littéraux, 4 clauses à 3 littéraux
- **HADD** (1 AND, 1 XOR) : 2 clauses à 2 littéraux, 5 clauses à 3 littéraux
- **ADD** (2 AND, 1 OR, 2 XOR) : 6 clauses à 2 littéraux, 11 clauses à 3 littéraux

En combinant ces données avec la structure en [étages](#) du circuit décrite au chapitre 2, il est alors possible de déduire la répartition des clauses en fonction de  $k$ .

## 6.2 Taille totale et ratio clause/variable

| Taille $p$ et $q$<br>(bits) | #Vars | #Clauses | #Clauses<br>2-littéraux | #Clauses<br>3-littéraux | Ratio<br>clauses/vars |
|-----------------------------|-------|----------|-------------------------|-------------------------|-----------------------|
| 5                           | 120   | 375      | 150                     | 215                     | 3.12                  |
| 10                          | 540   | 1750     | 700                     | 1030                    | 3.24                  |
| 15                          | 1260  | 4125     | 1650                    | 2445                    | 3.27                  |
| 20                          | 2280  | 7500     | 3000                    | 4460                    | 3.29                  |
| 25                          | 3600  | 11875    | 4750                    | 7075                    | 3.30                  |
| 30                          | 5220  | 17250    | 6900                    | 10290                   | 3.30                  |
| 35                          | 7140  | 23625    | 9450                    | 14105                   | 3.31                  |
| 40                          | 9360  | 31000    | 12400                   | 18520                   | 3.31                  |
| 45                          | 11880 | 39375    | 15750                   | 23535                   | 3.31                  |
| 50                          | 14700 | 48750    | 19500                   | 29150                   | 3.32                  |
| 55                          | 17820 | 59125    | 23650                   | 35365                   | 3.32                  |
| 60                          | 21240 | 70500    | 28200                   | 42180                   | 3.32                  |
| 65                          | 24960 | 82875    | 33150                   | 49595                   | 3.32                  |
| 70                          | 28980 | 96250    | 38500                   | 57610                   | 3.32                  |

Table 1: Évolution du nombre de variables et de clauses selon la taille des entrées  $p$  et  $q$

Le tableau 1 présente l'évolution du nombre de variables et de clauses dans les formules générées, en fonction de la taille en bits des opérandes  $p$  et  $q$ . On observe une croissance quadratique du nombre total de clauses, ainsi que du nombre de clauses à 2 et 3 littéraux, conformément aux équations établies au point précédent.

Le nombre de variables suit lui aussi une tendance quadratique. Cependant, il convient de distinguer les variables structurelles — introduites par la transformation de Tseytin — des variables significatives du problème. En effet, seules  $4 \cdot k$  variables correspondent directement aux bits des trois entiers  $p$ ,  $q$ , et  $x$  (deux facteurs et leur produit). Les milliers de variables restantes sont des variables auxiliaires utilisées pour encoder les connexions logiques entre les portes du circuit.

À cela s'ajoutent  $2 \cdot k$  clauses unitaires (non incluses dans le tableau) utilisées pour contraindre la sortie  $x$  à prendre la valeur spécifique à laquelle on s'intéresse.

Enfin, la dernière colonne du tableau met en évidence l'évolution du ratio entre le nombre de clauses et le nombre de variables. Ce ratio constitue un indicateur important pour anticiper la complexité d'une formule SAT. Nous en discuterons plus en détail dans la section suivante.



Les tendances évoquées ci-dessus sont confirmées visuellement dans le graphe ci-dessous, qui illustre l'évolution du nombre de variables, du nombre de clauses, ainsi que du ratio clauses/variables, en fonction de la taille des opérandes  $p$  et  $q$  :

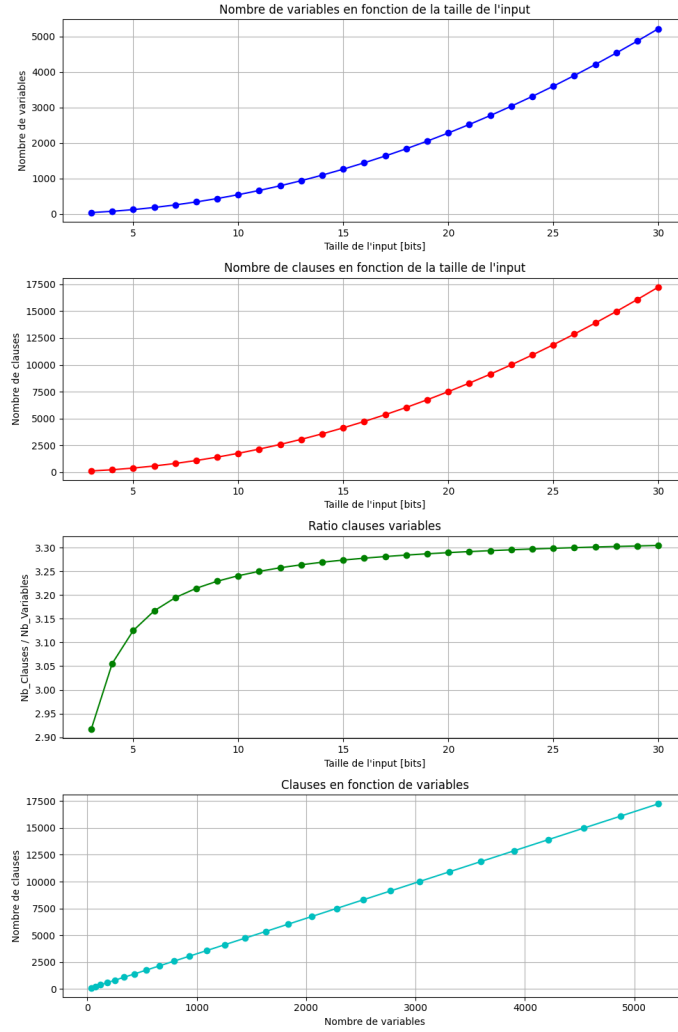


Figure 6: Évolution du nombre de variables, de clauses, et du ratio clauses/-variables en fonction de la taille des opérandes

### 6.3 Analyse du ratio clause/variable

Comme évoqué précédemment, le ratio clauses/variables constitue un bon indicateur préliminaire pour estimer la difficulté de résolution d'une formule SAT.

Dans sa thèse, Adam Joseph Beacham [Bea02] met en évidence un seuil critique situé autour de 4.3, au-delà duquel plus de 50 % des formules **aléatoires 3-SAT** deviennent insatisfiables.

Afin d’observer empiriquement ce phénomène, nous avons reproduit l’expérience sur des formules aléatoires 3-SAT. Pour chaque ratio allant de 3.5 à 5.0 (par pas de 0.1), nous avons généré 1000 instances contenant exactement 200 variables. Les clauses sont construites en sélectionnant uniformément au hasard trois variables distinctes parmi les 200, chacune étant mise négativement avec probabilité 1/2.

Les résultats sont présentés ci-dessous. Le premier graphe montre le nombre moyen de *backjumps* requis pour résoudre les formules, tandis que le second indique le taux de satisfiabilité (proportion d’instances SAT) pour chaque ratio testé.

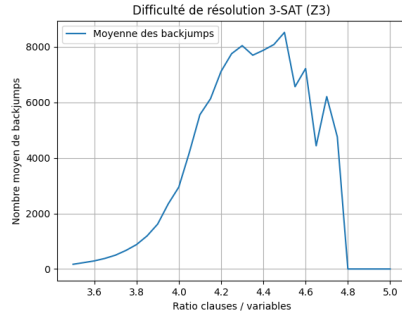


Figure 7: Nombre moyen de backjumps pour 1000 formules aléatoires 3-SAT à 200 variables en fonction du ratio clauses/variables.

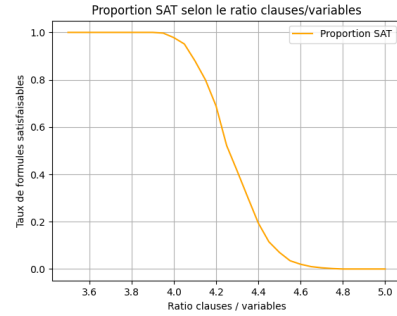


Figure 8: Taux de satisfiabilité en fonction du ratio clauses/variables.

On observe une chute nette du taux de satisfiabilité aux alentours du ratio 4.3, confirmant ainsi le comportement décrit par Beacham. Cette transition est également marquée par un pic d’effort algorithmique : dans l’intervalle [4.2, 4.7], le nombre de backjumps atteint des valeurs particulièrement élevées. Cela suggère que les formules situées dans cette zone sont les plus difficiles à résoudre pour le solveur, qu’elles soient satisfiables ou non. Le solveur semble rencontrer sa difficulté maximale dans cette zone, où décider si une formule est SAT ou UNSAT devient particulièrement coûteux.

Bien que nos formules ne soient ni aléatoires, ni exclusivement composées de clauses à 3 littéraux, nous avons souhaité vérifier si un comportement similaire à celui observé pour les formules aléatoires pouvait se manifester dans notre cas. En effet, nos formules contiennent à la fois des clauses à 2 et à 3 littéraux, mais leur structure reste proche du modèle 3-SAT selon certaines définitions.

Pour explorer cette question, nous avons généré aléatoirement des formules hybrides (2-SAT/3-SAT) présentant exactement le même nombre de variables,

ainsi que la même répartition entre clauses à 2 et à 3 littéraux que nos formules issues de la modélisation du problème FACT.

Les résultats sont sans appel : l'ensemble des instances générées se sont révélées insatisfiables (UNSAT), suggérant que notre structure particulière joue un rôle crucial dans le fait que nos formules soient, elles, satisfaisables malgré la même composition.

Ce constat met en évidence le caractère très peu probable, voire exceptionnel, d'une telle satisfiabilité lorsque la structure est aléatoire. En effet, avec un nombre aussi élevé de clauses — dont une majorité sont courtes (à 2 ou 3 littéraux) — la probabilité qu'une contradiction apparaisse quelque part dans la formule est naturellement très forte : il suffit qu'une seule clause soit impossible à satisfaire pour rendre l'entière de la formule UNSAT.

Autrement dit, nos formules satisfaisables semblent appartenir à une région très restreinte de l'espace des instances possibles. Une étude probabiliste plus rigoureuse permettrait sans doute de quantifier cette rareté, et d'appuyer l'idée que la structure déterministe issue de notre modélisation impose des contraintes spécifiques évitant certaines combinaisons "mortels" entre clauses.

## 7 Réduction de la taille de la formule

### 7.1 Règles de simplification élémentaires

Une première manière de procéder pour réduire la taille de nos formules a tout simplement été d'essayer d'y appliquer des règles de simplification booléenne.

Nous avons implémenté trois règles de simplification, directement inspirées de celles proposées sur le site *CNF Generator for Factoring Problems* [Sab03], utilisé comme référence pour la génération de nos formules. Elles permettent de diminuer le nombre de clauses et/ou la taille de certaines d'entre elles, tout en conservant l'ensemble des variables initiales : aucune variable n'est retirée du vocabulaire de la formule.

Ces règles sont appliquées de manière récursive, jusqu'à ce qu'elles n'aient plus d'effet sur la formule. Ce sont les suivantes :

1. **Suppression des clauses subsumées** : si les littéraux d'une clause  $A$  forment un sous-ensemble de ceux d'une clause  $B$ , alors  $B$  peut être supprimée.
2. **Réduction conditionnelle d'un littéral par parent subsumé** : si une clause  $A$  est de la forme  $(x, l_1, \dots, l_n)$  et qu'une clause  $B$  contient son complément, c'est-à-dire  $(\neg x, l_1, \dots, l_n, y_1, \dots, y_m)$ , alors le littéral  $\neg x$  peut être retiré de  $B$ .
3. **Substitution en cas d'égalité logique** : si deux clauses  $A = (l_1, \neg l_2)$  et  $B = (\neg l_1, l_2)$  sont présentes, alors toutes les occurrences de  $l_2$  dans les autres clauses peuvent être remplacées par  $l_1$  (à l'exception des clauses  $A$  et  $B$ ).

Après l'application de ces règles à nos formules, seule la première s'est révélée efficace. En effet, une fois la valeur de  $z$  fixée dans la formule, un certain nombre de clauses unitaires (ne contenant qu'un seul littéral) apparaissent. La première règle permet alors de supprimer toutes les clauses contenant ces littéraux (ou leur négation, selon le cas), via un mécanisme de propagation.

L'absence d'effet des deux autres règles a été confirmée par une analyse structurelle du circuit logique et des formules issues de la transformation de Tseytin. Une justification, plus ou moins formelle, est présentée ci-dessous :

**Justification de l'absence d'application des règles de simplification dans un circuit composé de transformées de Tseytin**

Pour commencer, nous pouvons reformuler les trois règles de simplification lorsqu'elles s'appliquent à des clauses de deux ou trois littéraux. Les motifs à rechercher dans les formules sont les suivants :

**Règle 1.**  $(l_1 \vee l_2) \wedge (l_1 \vee l_2 \vee l_3) \equiv (l_1 \vee l_2)$

**Règle 2.**  $(x \vee l_2) \wedge (\neg x \vee l_2 \vee l_3) \equiv (x \vee l_2) \wedge (l_2 \vee l_3)$

**Règle 3.**  $(x \vee \neg y) \wedge (\neg x \vee y)$

Dans tous les cas, on remarque qu'il faut que deux clauses partagent au moins deux variables pour qu'une règle puisse être appliquée, et qu'au moins l'une d'entre elles contienne exactement deux littéraux.

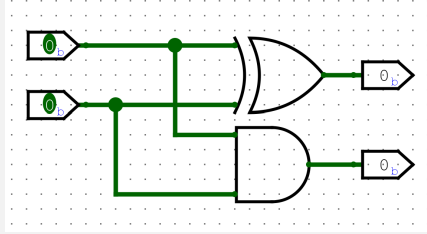
Les trois opérations logiques utilisées dans le circuit sont : *AND*, *OR* et *XOR*. Leurs transformations de Tseytin sont données ci-dessous :

*AND*  $C = A \cdot B$ :  $(\neg A \vee \neg B \vee C) \wedge (A \vee \neg C) \wedge (B \vee \neg C)$

*OR*  $C = A + B$ :  $(A \vee B \vee \neg C) \wedge (\neg A \vee C) \wedge (\neg B \vee C)$

*XOR*  $C = A \oplus B$ :  $(\neg A \vee \neg B \vee \neg C) \wedge (\neg A \vee B \vee C) \wedge (A \vee \neg B \vee C) \wedge (A \vee B \vee \neg C)$

Au sein même de ces formules, les [règles de simplification](#) ne sont pas directement applicables. Il faut donc rechercher dans le circuit des motifs structurels correspondant aux cas définis plus haut. Un seul motif satisfait la condition de partage de deux variables entre deux clauses, à savoir le suivant :



Extrait du circuit où le AND et le XOR partagent les mêmes entrées

Ce motif, où le *AND* et le *XOR* partagent les mêmes entrées, apparaît à plusieurs reprises. Il permet effectivement d'obtenir deux ensembles de clauses logiques faisant intervenir les mêmes variables, mais via des transformations de Tseytin différentes.

Nous examinons donc la transformée de Tseytin d'un *XOR* et d'un *AND* utilisant les mêmes entrées  $x$  et  $y$  :

$$\begin{aligned}
 &(\neg x \vee \neg y \vee s_{\text{AND}}) \wedge (x \vee \neg s_{\text{AND}}) \wedge (y \vee \neg s_{\text{AND}}) \\
 &\quad \wedge \\
 &(\neg x \vee \neg y \vee \neg s_{\text{XOR}}) \wedge (\neg x \vee y \vee s_{\text{XOR}}) \\
 &\quad \wedge (x \vee \neg y \vee s_{\text{XOR}}) \wedge (x \vee y \vee \neg s_{\text{XOR}})
 \end{aligned}$$

Parmi ces clauses, seules celles issues de la transformée du *AND* contiennent deux littéraux :  $(x \vee \neg s_{\text{AND}})$  et  $(y \vee \neg s_{\text{AND}})$ . Ces clauses ne correspondent pas au motif de la [règle 3](#), et la clause à trois littéraux associée à ce *AND* ne permet pas davantage l'application des autres règles.

Il reste alors à étudier les combinaisons possibles avec les quatre clauses issues du *XOR*. On observe que dans chaque paire possible de ces clauses, une seule variable est partagée, ce qui est insuffisant pour appliquer les [règles 1](#) ou [2](#), qui nécessitent le partage d'au moins deux variables.

On peut donc conclure que les [règles de simplification](#) ne sont pas applicables, en l'état, à notre implémentation spécifique.

Il est par ailleurs important de noter que, même lorsque la [première règle](#) est applicable, son effet reste limité : elle ne permet pas de transformer la structure de la formule de manière à rendre, par récursivité, l'une des deux autres règles applicable. Autrement dit, son application ne débloque pas l'usage des [règles 2](#) ou [3](#).

## 7.2 Propagation unitaire

Après l'échec de l'application des règles de simplification classiques présentées précédemment, nous avons exploré une autre méthode inspirée des solveurs SAT modernes : la *propagation unitaire*. Cette règle est un élément fondamental des algorithmes de type DPLL [[Wik25b](#)] [[DLL62](#)] et CDCL; ces algorithmes sont

notamment présentés en détail dans le chapitre 2.2 du livre de Kroening et Strichman [KS08].

**Règle de propagation 1** *Soit ( $l$ ) une clause unitaire, composée d'un seul littéral  $l$ . Alors :*

- *toute clause contenant  $l$  peut être supprimée de la formule ;*
- *dans toute clause contenant  $\neg l$ , le littéral  $\neg l$  peut être retiré.*

Nous avons implémenté cette règle dans notre code, disponible sur Git-Lab [Bar25]. Elle permet de simplifier considérablement les formules en supprimant de nombreuses clauses redondantes ou devenues triviales.

Dans notre cas, les clauses unitaires apparaissent naturellement dès le début, car elles correspondent aux bits du produit  $z = p \cdot q$  que nous fixons explicitement. Si  $p$  et  $q$  sont codés sur  $l$  bits, alors  $z$  est typiquement représenté sur  $2l$  bits. Cela signifie que l'on dispose d'office de  $2l$  clauses unitaires à appliquer.

Nous avons observé que chaque clause unitaire peut entraîner, par effet de propagation, la simplification de plusieurs autres clauses. Une étude plus précise de cette propagation reste à effectuer, notamment en mesurant le nombre moyen de clauses supprimées, ainsi que son impact sur des indicateurs tels que le temps de résolution ou le nombre de backjumps. Cependant, on peut s'attendre à une diminution significative de ces deux métriques.

### 7.3 Réduction par assignation de variables

Dans cette dernière expérimentation, nous avons étudié l'effet d'une assignation anticipée de certaines variables sur la taille et la complexité de la formule SAT. L'idée est simple : fixer manuellement la valeur d'une ou plusieurs variables internes du circuit dans le but de supprimer ces variables de la formule.

Cette assignation déclenche mécaniquement une application de la règle de propagation unitaire, identique à celle présentée dans la section précédente, mais cette fois appliquée à des variables qui ne faisaient pas initialement partie du produit cible  $z = p \cdot q$ . On espère ainsi que la formule se simplifie significativement, tout en restant difficile à résoudre.

Dans cette première approche, les variables ont été choisies aléatoirement parmi l'ensemble des variables internes. Il aurait été intéressant d'exploiter la connaissance structurelle du circuit pour cibler des variables spécifiques — par exemple en fonction de leur position ou de leur rôle logique — mais cette analyse plus fine n'a pas pu être menée, faute de temps.

Les résultats expérimentaux sont présentés ci-dessous. Ils montrent l'évolution du temps de résolution et du nombre de backjumps en fonction du nombre de variables fixées :

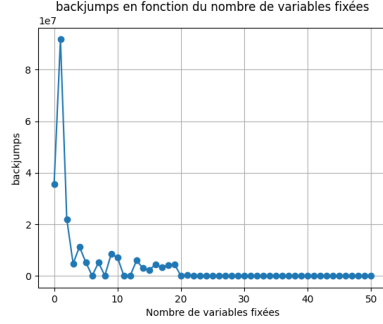


Figure 9: Nombre de backjumps en fonction du nombre de variables fixées.

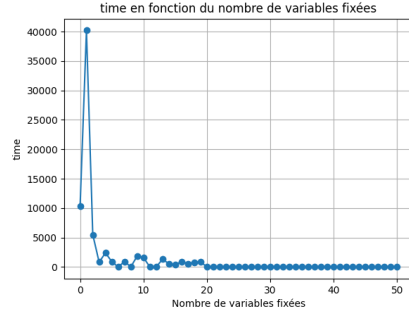


Figure 10: Temps de résolution en fonction du nombre de variables fixées.

Les formules utilisées ici correspondent à une multiplication de deux entiers codés sur 30 bits, ce qui génère 5220 variables et 17 250 clauses. On observe rapidement une baisse marquée de la complexité : dès que deux variables sont fixées, le nombre de backjumps et le temps de résolution sont réduits d'un facteur 3 à 4. Cette tendance s'accroît avec plus de variables fixées : vers 20 variables fixées, la résolution devient quasi-instantanée.

Cela suggère que la fixation de variables a un impact majeur sur la structure du problème, probablement parce qu'elle supprime un grand nombre de clauses courtes. Une estimation brute indique qu'une variable fixée permettrait d'éliminer entre 4 et 5 clauses en moyenne, ce qui contribue à désengorger fortement la structure CNF.

Enfin, nous avons remarqué un phénomène surprenant : la fixation de la première variable peut parfois augmenter significativement le nombre de backjumps — jusqu'à un facteur six dans certains cas. Ce comportement inattendu suggère que l'ordre des clauses ou des littéraux dans la formule pourrait influencer la manière dont le solveur explore l'espace de recherche.

Pour explorer cette hypothèse, nous avons mené une petite expérience : une même formule a été sélectionnée, puis perturbée de différentes manières en réorganisant aléatoirement l'ordre des clauses, ainsi que l'ordre des variables à l'intérieur de chaque clause. L'idée était d'observer si cette simple transformation syntaxique, sans aucune modification logique, pouvait affecter le comportement du solveur.

Les résultats sont présentés ci-après :

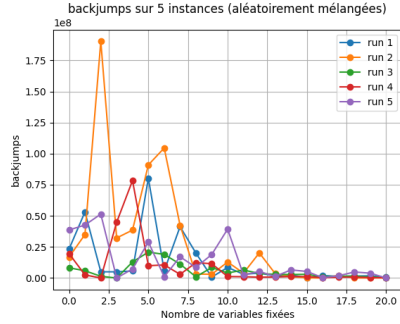


Figure 11: Évolution du nombre de backjumps selon le nombre de variables fixées pour plusieurs ordres aléatoires.

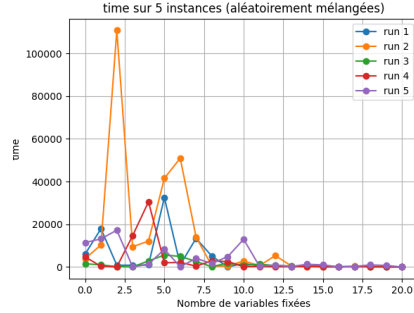


Figure 12: Évolution du temps de résolution pour les mêmes variantes.

On observe clairement que l'ordre des clauses et des variables a un impact mesurable sur la performance du solveur, tant en termes de temps qu'en termes de backjumps. Certaines versions de la formule conduisent à une simplification bien plus rapide, tandis que d'autres ralentissent significativement la résolution.

Ces observations ouvrent la voie à une piste intéressante : explorer si un réordonnancement stratégique des clauses pourrait permettre de conserver une complexité élevée, même après simplification. Cela pourrait constituer une méthode complémentaire pour construire des instances à la fois compactes et difficiles à résoudre — notre objectif initial.

## 8 Conclusion

Au cours de ce travail, nous avons exploré différentes approches dans le but de générer des formules SAT à la fois compactes et difficiles à résoudre. Bien que notre objectif initial — obtenir des instances à la fois petites et particulièrement exigeantes pour les solveurs — n'ait pas été atteint de manière pleinement satisfaisante, plusieurs enseignements utiles ont émergé.

D'une part, plusieurs pistes théoriques et expérimentales ont pu être écartées, ce qui nous a permis de mieux cibler les approches réellement pertinentes. D'autre part, certaines idées se sont révélées prometteuses. Par exemple, la simplification par propagation unitaire ou par assignation de variables permet de réduire la taille des formules, même si cela rend parfois leur résolution plus facile.

Nous avons également observé des effets moins attendus : l'ordre des clauses ou des variables peut influencer la complexité de résolution, ce qui montre que l'organisation interne de la formule joue un rôle non négligeable. Enfin, nos tests ont mis en évidence le caractère très particulier de nos formules : alors que des formules aléatoires ayant une structure similaire sont toujours insatisfaisables,



les nôtres restent satisfiables. Cela suggère que leur structure précise — issue de la modélisation logique de la multiplication — les rend à la fois rares et intéressantes à étudier.

Parmi les prolongements envisageables, on pourrait tester nos méthodes de simplification sur des formules de taille bien plus grande, afin de déterminer à quel point il est possible de fixer des variables sans rendre la résolution triviale. Ce type d'expérimentation se heurte néanmoins à des limites pratiques : le temps nécessaire pour résoudre des instances générées avec des opérandes de plus de 28 bits dépasse déjà plusieurs heures, rendant toute étude systématique difficile à mener à court terme.

Enfin, l'une des difficultés majeures rencontrées réside dans la définition même de ce qu'est une "formule difficile". Faut-il se baser sur le temps de résolution, le nombre de backjumps, ou d'autres métriques ? Et à partir de quel seuil une formule devient-elle "trop facile" ou "suffisamment difficile" ? Ces questions restent ouvertes, et constituent un champ de recherche en soi.

Ce projet, bien qu'axé sur des expérimentations concrètes, a ainsi permis de dégager des pistes de réflexion plus larges sur la structure des formules SAT et sur la manière d'en manipuler la difficulté.

## References

- [DLL62] Martin Davis, George Logemann, and Donald Loveland. “A machine program for theorem-proving”. In: *Communications of the ACM* 5.7 (1962), pp. 394–397.
- [MSL+92] David Mitchell, Bart Selman, Hector Levesque, et al. “Hard and easy distributions of SAT problems”. In: *Aaai*. Vol. 92. 1992, pp. 459–465.
- [HHW96] Tad Hogg, Bernardo A. Huberman, and Colin P. Williams. “Phase transitions and the search problem”. In: *Artificial Intelligence* 81.1 (1996). Frontiers in Problem Solving: Phase Transitions and Complexity, pp. 1–15. ISSN: 0004-3702. DOI: [https://doi.org/10.1016/0004-3702\(95\)00044-5](https://doi.org/10.1016/0004-3702(95)00044-5). URL: <https://www.sciencedirect.com/science/article/pii/0004370295000445>.
- [HW97] Satoshi Horie and Osamu Watanabe. “Hard instance generation for SAT”. In: *Algorithms and Computation: 8th International Symposium, ISAAC’97 Singapore, December 17–19, 1997 Proceedings* 8. Springer. 1997, pp. 22–31.
- [Bea02] Adam Joseph Beacham. *The complexity of problems without backbones*. National Library of Canada= Bibliothèque nationale du Canada, Ottawa, 2002.
- [Sab03] Amr Sabry. *From Boolean Circuits to CNF*. <https://cgi.luddy.indiana.edu/~sabry/cnf.html>. Accessed: 2025-06-07. 2003.
- [WGS03] Ryan Williams, Carla P Gomes, and Bart Selman. “Backdoors to typical case complexity”. In: *IJCAI*. Vol. 3. 2003, pp. 1173–1178.
- [Kil+05] Philip Kilby et al. “Backbones and backdoors in satisfiability”. In: *AAAI*. Vol. 5. 2005, pp. 1368–1373.
- [Ans+08] Carlos Ansótegui et al. “Measuring the Hardness of SAT Instances.” In: *AAAI*. Vol. 8. 2008, pp. 222–228.
- [KS08] Daniel Kroening and Ofer Strichman. *Decision procedures*. Vol. 5. Springer, 2008.
- [Jär+12] Matti Järvisalo et al. “Relating proof complexity measures and practical hardness of SAT”. In: *International Conference on Principles and Practice of Constraint Programming*. Springer. 2012, pp. 316–331.
- [BY13] Joseph Bebel and Henry Yuen. “Hard SAT instances based on factoring”. In: *SAT Competition* (2013), p. 102.
- [Oh15] Chanseok Oh. “Between SAT and UNSAT: the fundamental difference in CDCL SAT”. In: *International Conference on Theory and Applications of Satisfiability Testing*. Springer. 2015, pp. 307–323.

- [Li+23] Yang Li et al. “Hardsatgen: Understanding the difficulty of hard sat formula generation and a strong structure-hardness-aware baseline”. In: *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2023, pp. 4414–4425.
- [MB24] Leonardo de Moura and Nikolaj Bjørner. *Z3 Theorem Prover*. <https://github.com/Z3Prover/z3>. 2024.
- [Bar25] Loïc Bardi. *Projet Bachelor — Génération et Analyse de Formules SAT*. GitLab repository. <https://gitlab.unige.ch/Loic.Bardi/projetbachelor>. 2025.
- [Wik25a] Wikipedia. *Boolean satisfiability problem* — *Wikipedia, The Free Encyclopedia*. <http://en.wikipedia.org/w/index.php?title=Boolean%20satisfiability%20problem&oldid=1297184619>. [Online; accessed 29-June-2025]. 2025.
- [Wik25b] Wikipedia. *DPLL algorithm* — *Wikipedia, The Free Encyclopedia*. <http://en.wikipedia.org/w/index.php?title=DPLL%20algorithm&oldid=1292217523>. [Online; accessed 29-June-2025]. 2025.
- [Wik25c] Wikipedia. *Tseytin transformation* — *Wikipedia, The Free Encyclopedia*. <http://en.wikipedia.org/w/index.php?title=Tseytin%20transformation&oldid=1225425428>. [Online; accessed 29-June-2025]. 2025.